Mini: Project documentation
Stage 05

Karel Šafr

1 Your project: stage 05

You will implement inner-to-outer name lookup and connect it to variable reads, grouped expressions and assignment targets and values. Return existing declaration IDs, preserve shadowing and reject an unfinished nearest binding without fallback. Declaration allocation and block traversal are already complete. Bare literals still run while the new binding visitors are unfinished.

Use the start version for your own work. The table below shows what already works and what your change will add. Follow the exercise in order, then compare your results with the solution when it is provided.

Area	**Start**	**After completion**
Syntax and AST	Literals, grouping, variables, assignments and blocks are parsed.	Unchanged.
Declarations and blocks	Declaration IDs, definition flags and scope entry/visit/exit are supplied.	Unchanged.
Bare literal execution	Works; literals need no name resolution.	Unchanged.
Variable reads and grouping	Explicit TODOs stop their expression traversal.	Reads receive IDs; grouping recursively visits its owned child.
Assignment	Its visitor reaches an explicit TODO.	Bind the declared target, then resolve its whole value expression.
Lexical lookup	The helper itself is a TODO.	The nearest matching active scope supplies its existing ID.
Invalid names	Name uses stop at the caller TODO.	Undefined nearest bindings and completely absent names produce distinct errors.
Runtime and grammar	Storage is provided; operators and general calls are absent.	No new runtime component or grammar is implemented.

How to use this document

Begin with ASSIGNMENT.txt in this folder. Run the starting example, read the task, and then use this guide to trace the relevant code. The file reference explains all 31 source and configuration files. When I provide the solution, its READ_ME.txt will help you compare your approach with the worked solution.

You have everything you need for this exercise in the folder you received: the compiler source, runtime, build configuration, examples and checks. Our compiler translates Mini to C++17 and then uses a C++ compiler to produce a native executable. Follow these separate steps when you explain how a Mini program runs.

2 Build the compiler and make a first input

Requirements: CMake 3.20 or newer, a C++17 compiler and its standard library. The commands below use a macOS shell and Unix Makefiles. Run them in a writable working copy of the selected package, in the directory containing CMakeLists.txt. The project does not download third-party C++ libraries.

```
cmake --version
c++ --version
cmake -S . -B build -G "Unix Makefiles" \
  -DCMAKE_BUILD_TYPE=Debug
cmake --build build --parallel 2
./build/minic --help
```

The supplied inputs/ directory contains starting_point.mini, acceptance.mini and edge cases. Save this additional manual demonstration as inputs/hello.mini:

```
fun main() {
  print 42;
  print "Mini";
}
```

```
./build/minic tokens inputs/hello.mini
./build/minic ast inputs/hello.mini
./build/minic run inputs/hello.mini
```

The run command prints 42 and Mini on separate lines and exits 0 in both the untouched start and completed solution. This demonstration avoids the exercise TODOs.

Run the supplied baseline and checkpoint checks

```
./build/minic run inputs/starting_point.mini
python3 CHECK_PROJECT.py --mode start --compiler ./build/minic
# After editing and rebuilding:
python3 CHECK_PROJECT.py --mode solution --compiler ./build/minic
```

The baseline works before editing. Python 3.8+ is needed only for the optional checker. PASS includes matched expected errors; start-mode PASS does not mean completion. Rebuild after C++ edits, using a separate build for each stage. Later examples abbreviate ./build/minic as minic; substitute your executable path. ASSIGNMENT.txt lists the bundled inputs; other manual examples may need to be saved separately.

3 Architecture: calls, data and two builds

Order / data	Responsible component
1. SourceText	source.cpp
2. tokens	lexer.cpp
3. parsed AST	parser.cpp
4. core AST	lower.cpp
5. resolved AST	resolver.cpp
6. C++ text	codegen.cpp
7. native program	HostCompiler

driver.cpp coordinates the path; build/run add the external C++ compiler.

main.cpp is the process boundary; cli.cpp validates arguments. SourceText::fromFile reads bytes before executeCommand drives the shared pipeline. A command selects how far to proceed; it does not select an independent parser.

The two compilations are different

- CMake builds our C++ implementation into the mini_compiler static library and the minic executable. configure_file generates mini_build_config.hpp in the build directory.
- minic cpp emits your Mini program as C++ text. minic build/run invokes HostCompiler to compile that generated translation unit with the header-only runtime. The resulting program is a different executable.
- The runtime executes Value operations and printing when that generated executable runs. It is not the lexer, parser or resolver, and it is not linked as a separate runtime library.

Current tree and pass contracts

Program.main_body holds the statements in the single main function. Resolution is an empty result type. Name bindings are stored as SymbolId annotations in AST nodes, not in the empty result object.

For this stage, lowering is an identity pass. It is visible in the driver so the same pipeline can grow without changing the CLI.

SourceText owns the source string. Tokens own lexeme/payload data and retain source spans. AST variants encode node kinds; unique_ptr owns recursive children. Moves transfer ownership between parser/lowering results. This compiler memory is distinct from scope tables and from values in a running Mini program.

4 Commands, output files and errors

Command	Last phase / observable result
tokens FILE	scan; prints the token sequence including EOF. A fragment may be enough.
ast FILE	parse; prints the parsed tree before lowering and resolution.
lower FILE	lower; prints the core tree before resolution.
check FILE	resolve; success is silent, status 0. No C++ is emitted or executed.
cpp FILE [-o OUT.cpp]	emitCpp; writes text to stdout or to the given file.
build FILE [-o OUT]	host C++ compilation; creates an executable but does not run it.
run FILE	host C++ compilation then execution; uses and cleans a temporary directory.
--help	CLI only; no input is read. For the available features, use the opening scope table.

Without -o, build chooses an executable beside the input (normally the stem, with the platform suffix). Choose an explicit output name when you want to keep the generated file. cpp/build reject overwriting the Mini input. run forwards the generated program output and its exit status; it does not leave a reusable executable.

```
./build/minic cpp inputs/hello.mini -o build/hello.cpp
./build/minic build inputs/hello.mini -o build/hello
./build/hello
```

Status / phase	Interpretation
0	Requested operation succeeded; check/build can legitimately print nothing.
1 / lex, parse, resolve, codegen, io, backend	Controlled compiler error; inspect its phase and location where provided.
1 / runtime	Generated program failed during an operation; e.g. incompatible operands.
2 / cli	Wrong command/arguments. No-argument minic is not the help command.
70 / internal	Unexpected std::exception. The stage08 lowering TODO deliberately uses this path.

check is not a dynamic type checker and cannot prove that execution will succeed. A parsed program can still fail resolution; a resolved program can still fail code generation at a TODO or fail later at runtime. Follow the earliest failing phase.

5 Stage 05: lexical lookup is a compiler operation

This exercise builds on block scopes: once several maps are active, how do we find the declaration used by a particular name? Keep that question separate from runtime memory. The resolver connects names to identities; generated C++ variables hold the values when your program runs.

Your project retains the completed declaration protocol and block traversal. You implement resolve Name(), the whole resolveExpression() visitor and the AssignmentStatement branch body. The start deliberately stops reads, grouping and assignments at their caller TODOs, so even local reads

wait for this work. Bare literals and unused declarations with bare literal initializers remain valid. A parenthesized literal still reaches the grouping TODO.

Three kinds of data that must not be confused

Data	Purpose and lifetime
Token / AST	Compiler representation of source spelling and structure; literals carry source values.
Scope map / Binding	Compiler metadata connecting a name to an id and a defined flag during resolution.
Runtime Value	Number, bool, string or nil stored and used when generated C++ executes.

The scope stack is a vector of maps. Its last map represents the most recently entered block. A declaration receives a new SymbolId once; reads and assignment targets must receive that same identity. These IDs distinguish declarations within one compilation, not persistent objects across changed source files. An assignment updates a runtime value later; it does not allocate another declaration ID.

VariableExpression, VarStatement and AssignmentStatement contain optional resolved_id fields. The parser leaves these empty. Supplied declare() assigns IDs to declarations; your variable-read and assignment visitors store lookup results on the actual AST nodes. Code generation consumes these annotations to choose names such as mini_variable_0. Grouping carries no declaration ID of its own but must expose its child to traversal.

Program still contains main_body, and Resolution is still empty: there are no general function declarations, parameter bindings or FunctionId values in this package. Lowering is still identity. The grammar remains one main with literals, grouping, print, variables, assignment and blocks. Operators, branches, loops and general calls arrive in later prepared starts.

6 Connect lexical lookup to reads, grouping and assignment

Edit three parts of src/resolver.cpp: resolveName(), the entire resolveExpression() body and the AssignmentStatement branch body. Keep signatures and the assignment dispatch condition. Leave declare(), define(), the declaration visitor, block traversal, AST ownership and runtime unchanged. Your methods connect uses to IDs already allocated by declarations.

Search active maps from inside out

- In resolveName(), reverse-iterate scopes_. A missing name in one map means continue outward, not an immediate unknown-name error.
- At the first match, reject defined == false immediately. Do not skip that binding and use an outer declaration with the same spelling.
- Otherwise return the existing id. Only exhausting all maps without a match produces the unknown-variable error. Do not modify maps, allocate IDs or return values.

Condition	Resolve diagnostic message fragment
Nearest match is not defined	cannot read variable '<name>' in its own initializer
No active scope contains the name	unknown variable '<name>'

Both errors use Phase::Resolve, source_.name() and the supplied span. The main program formats path, position and phase on stderr and returns 1. The supplied declaration visitor inserts Binding{id, false} before resolving its initializer, so an unfinished inner name already shadows an outer name. Its flag records readiness for a use, not a runtime value or a static type.

Visit the actual expression nodes

In resolveExpression(), obtain mutable access to a VariableExpression, call resolveName with its name and expression.span, store the result in resolved_id and return. For GroupingExpression, recursively resolve its owned child and return. Literals fall through unchanged. Do not annotate a copied node: the generator must see the annotation in the original tree. Parentheses cannot hide an unresolved name.

Resolve both sides of an assignment

In the AssignmentStatement branch, bind the target with its name and name_span and store the returned ID in assignment->resolved_id. Then call resolveExpression on assignment->value and return. A valid target is not enough: the value may contain a grouped read or an unknown name. Assignment does not create a declaration; it reuses the existing target ID.

These three parts share one lookup policy. Searching outermost-first breaks shadowing; falling back past an undefined match breaks initialization rules; skipping the assignment value leaves names unchecked. The bundled cases isolate these mistakes without adding runtime memory work or changing syntax.

7 Worked outer assignment: one declaration, a changing value

Save this complete program as w05_outer.mini, including a final newline. ast and lower already work. The untouched start cannot bind the assignment: minic run w05_outer.mini exits 1 with empty stdout and "resolve: TODO WEEK05: bind assignment target and value" on stderr, preceded by the input path/location. check and cpp reach the same failure.

```
fun main() {
  var x = 10;
  { x = 30; }
  print x;
}
```

Both before and after the edit, minic ast w05_outer.mini gives the complete stdout below. lower gives the same text. These commands stop before resolve, and the formatter does not print resolved_ id fields:

```
(program
  (fun main ()
    (var x (number 10))
    (block
      (assign x (number 30))
    )
    (print (variable x))
  )
)
```

Hand trace of resolution, not an emitted trace

Occurrence	Search and identity
var x = 10	declare allocates ID 0; the literal initializer needs no lookup.
x = 30 inside the block	The inner map has no x; the outer map supplies ID 0.
print x after the block	The current main map supplies the same ID 0.

After completion and rebuild, minic check w05_outer.mini succeeds silently with exit 0. minic run w05_outer.mini prints exactly 30 followed by a newline and also returns 0. The program has one declaration, not an inner copy of x. The inner assignment changes the same storage that the later print reads.

minic cpp w05_outer.mini makes that distinction visible: the supplied generator uses mini_variable_0 for the declaration, assignment and read. Its temporary values capture initializer and assignment expressions; its braces retain block structure. At execution, that variable first holds 10 and then 30. No runtime value is calculated or stored in scopes_ while the resolver performs the hand-traced search.

The same program was deliberately rejected by stage 04's current-only lookup. Stage 05 changes the resolution rule without changing its tokens, grammar, AST shape or runtime representation. A successful check still does not establish that the external C++ toolchain can build the generated source; run exercises that later path too.

8 Check lookup boundaries and prepare for expressions

A working start baseline

```
fun main() { print 42; }
```

Create w05_literal.mini from that line. minic run w05_literal.mini prints 42 and returns 0 before and after your edit. The bundled inputs/starting_point.mini is identical. A bare literal requests no lookup, unlike grouping which deliberately reaches a traversal TODO in the start. Unused declarations with bare literal initializers can also be checked because their protocol is supplied.

A nearest match that must not be used

```
fun main() { var x = 1; { var x = x; } }
```

Create w05_self.mini and run minic check w05_self.mini. Before your edit, it reports "resolve: TODO WEEK05: bind variable read". Afterwards it exits 1 with "resolve: cannot read variable 'x' in its own initializer" and empty stdout. The inner binding exists but is undefined. Stop there; do not continue to outer x and copy its value 1.

Use the bundled cases to test traversal as well as search: edge_grouped_read.mini prints 7 through two parentheses and an outer lookup; edge_assignment_rhs.mini prints 2 after resolving its grouped value. Unknown targets and unknown right-hand names must both be rejected. A grouped initializer must resolve its child before the supplied declaration visitor marks the new name defined.

Completion criteria

- Return existing IDs for the nearest matching active declaration, and annotate the real read and assignment nodes. Recursively traverse every grouping child.

- Reject an undefined nearest match without fallback and reject completely unknown names. Preserve supplied duplicates, declaration allocation and scope pops.
- Explain why assignment resolves its target before its value, why the value still needs traversal, and why changing a runtime value does not change binding identity.

Keep your completed binding traversal. In Stage 07, we will implement the additive parser loop and unary operator branch. The next project supplies operator AST nodes, other parser levels, additional resolver traversal, generated operations and runtime helpers. Existing binding rules continue to apply inside larger expressions; division and string concatenation remain outside that exercise.

9 File reference: responsibilities

Use this table to locate the code in your project folder. The two counts give each file's total length in start and solution, including comments and blank lines. They are not line numbers to edit. The worked solution adds 25 nonblank code lines, excluding comments and the unchanged skeleton. Your implementation may use a different number of lines; follow the task and explain how your code produces the required behavior.

File	Total lines: start / solution	Responsibility and boundary
.gitignore	7 / 7	Version-control exclusions for generated build/output files. It is configuration, not a compiler phase.
CMakeLists.txt	57 / 57	Requires CMake 3.20 and C++17; configures mini_ build_config.hpp, builds mini_compiler and links minic. Run the bundled checkpoint checks separately.
runtime/mini_runtime.hpp	46 / 46	Header-only Value representation, printing and the operations available in this stage. Included by generated programs, not a replacement for the frontend.
src/ast.cpp	108 / 108	formatAst: readable structural dump shared by ast and lower. This is a view of the tree, not an evaluator or stable serialization format.
src/ast.hpp	97 / 97	Owns the stage-specific Program, Expression and Statement data structures. Tagged variants distinguish node kinds; unique_ptr owns recursive children.
src/build_config.hpp.in	15 / 15	CMake template for compiler path/family, runtime include directory and executable suffix. Its generated header belongs to the build directory.
src/cli.cpp	129 / 129	Validates command syntax and -o usage; formats common help. The final-language help paragraph does not promise all features in an early stage.
src/cli.hpp	37 / 37	CommandKind and CommandLine: command, input path and optional output path. Declares parse CommandLine and helpText.
src/codegen.cpp	197 / 197	Traverses the current tree, emits values/statements, and creates the host main wrapper. See the stage-specific sections for supported nodes and the exercise.
src/codegen.hpp	19 / 19	emitCpp(const Program&, const Resolution&) returns one C++17 translation unit. It consumes the resolved tree; it does not launch a compiler.

File	Total lines: start / solution	Responsibility and boundary
src/diagnostics.cpp	69 / 69	Maps diagnostic phases to labels and formats source-name/line/column when available. Supplies consistent compiler-side error messages.
src/diagnostics.hpp	49 / 49	Phase and CompileError contracts, optional source location and diagnostic formatting interface. Compile Error is distinct from RuntimeError.
src/driver.cpp	200 / 200	Runs phases in order and stops at the requested view. Writes C++, manages temporary directories and invokes HostCompiler for build/run.
src/driver.hpp	20 / 20	executeCommand(const CommandLine&, const Source Text&, ostream&) is the shared orchestration entry point used by the executable.
src/host_compiler.cpp	281 / 281	Selects and invokes the external C++ toolchain, quotes arguments, captures compilation logs and normalizes process status. Checks fresh executable output.
src/host_compiler.hpp	44 / 44	HostCompilerConfig and HostCompiler: configured toolchain, compile(source, executable, scratch), and run(executable).
src/lexer.cpp	246 / 246	Reads characters and creates tokens with spans and literal payloads, skipping whitespace/comments. Keyword recognition and grammar acceptance are separate.
src/lexer.hpp	19 / 19	scan(const SourceText&) returns vector<Token>. The public function hides the scanner cursor and token-building helpers.
src/lower.cpp	14 / 14	Provides the explicit lowering stage. Identity before stage08; from stage08 rewrites parsed for into core blocks and while after the exercise is completed.
src/lower.hpp	17 / 17	lower(Program) returns Program. Taking the tree by value supports moving its owned children instead of copying them.
src/main.cpp	41 / 41	Process entry: collects argv, parses CLI, loads Source Text and calls driver. Catches compiler errors and unexpected C++ exceptions at the boundary.
src/number_format.cpp	43 / 43	Implements locale-independent numeric formatting used by diagnostics/dumps and generated code; avoids decimal-comma differences.
src/number_format.hpp	17 / 17	Declares formatNumberForDisplay and formatNumber ForCpp. A human-readable number and a valid C++ double literal serve different purposes.
src/parser.cpp	229 / 229	Recursive-descent parser for the current stage. Constructs AST nodes and reports the first syntax error; does not execute expressions or resolve names.
src/parser.hpp	20 / 20	parse(const SourceText&, const vector<Token>&) returns Program. Source text supports diagnostics; tokens supply the grammar input.
src/resolver.cpp	134 / 148	Name-analysis pass: no-op in stage03, local/scope binding from stage04, outward lookup after stage05 and function resolution from stage09.
src/resolver.hpp	19 / 19	resolve(const SourceText&, Program&) returns Resolution and may annotate AST names. The exact result and lookup capabilities grow by stage.

File	Total lines: start / solution	Responsibility and boundary
src/source.cpp	77 / 77	Loads the input file, indexes line starts and maps offsets to source positions. Shared by lexical, syntax and semantic diagnostics.
src/source.hpp	44 / 44	SourceText owns source name, contents and line starts; SourceSpan stores offset, length, line and column. Declares file loading and span lookup.
src/token.cpp	119 / 119	formatTokens: prints token kinds, locations, lexemes and payloads. EndOfFile is explicit; token names describe scanning, not full language support.
src/token.hpp	66 / 66	TokenKind, TokenLiteral and Token. Each token owns lexeme text, a source span and an optional double/ string literal payload.

Use CHECK_PROJECT.py, checks/CASES.json and inputs/ for the checks described in ASSIGNMENT.txt. These support files are separate from the compiler sources listed here. A worked explanation accompanies the solution.

10 Working on your project

Keep the build and runtime together

Keep runtime/ beside src/. If you move the project, configure and build it again in its new location; the build records paths to the compiler and runtime. HostCompiler uses MINI_CXX if set, then CXX, then the compiler chosen by CMake. These overrides take one executable name or path, without extra flags.

Run the project in your IDE

In CLion, open the folder containing CMakeLists.txt and build the minic target. Set the program arguments to run inputs/starting_point.mini. Set the working directory to the project folder. Your IDE may use its own build directory; use the minic executable from that directory when following the command-line examples.

What you see	What to do
Cannot open input	Check the working directory and filename, including the .mini extension.
The old TODO still appears	Rebuild. Check that you are running minic from the copy you edited.
A backend error	Read the compiler log in the error. Check your C++ toolchain and the runtime path.
An unsupported feature	Check the opening scope table and the examples for this stage.
CTest finds no tests	Use CHECK_PROJECT.py or the manual commands in ASSIGNMENT.txt.

Before you finish

Your change in this exercise is in src/resolver.cpp. Rebuild, run the acceptance example and complete the edge cases in ASSIGNMENT.txt. The solution mode of CHECK_PROJECT.py checks your edited start as well as the worked solution.

- Show an example that works before your change, then explain the new behavior after your change.
- Point to the function you changed. Explain the order of its steps and why the edge cases matter.
- For a rejected example, identify the failing phase and explain why that error is expected.

Keep your completed work

Save this completed project separately. I will provide a new starting project for our next exercise. It will include the supporting changes needed for that topic. Open and build that new project as a whole; copying one file between versions can mix incompatible parts.